

WAYPOINT

Real-Time Fleet & Delivery Tracking Platform

4-WEEK TIMELINE & PROGRESS TRACKER

Document Version: 1.0

Prepared for: Raynald — Backend Engineering Portfolio Project

Date: July 2026

1. How This Timeline Is Structured

This timeline is intentionally NOT a copy of a standard "Week 1: Setup, Week 2: Core, Week 3: Async, Week 4: Polish" pattern. Waypoint's biggest risk is concentrated in one unfamiliar mechanism — real-time WebSocket communication — so the timeline is structured to retire that risk immediately in Week 1, rather than leaving it for later the way a typical CRUD project schedule would.

Week	Theme	Primary Risk Being Retired
Week 1	Prove the Mechanism + Core Domain	"Can I actually get real-time messaging working at all?"
Week 2	Merge Real-Time Into the Domain	"Can I connect real-time messaging to real, authorized, per-user data?"
Week 3	Operational Features & Hardening	"Does this behave correctly under failure and abuse, not just the happy path?"
Week 4	Testing, Deployment & Presentation	"Is this actually deployed, tested, and explainable to someone else?"

Use the checkboxes in each week's section to track your own progress. At the end of each week, fill in the Self-Check Gate before moving to the next week — do not proceed if a gate item is unmet without consciously deciding to descope something.

2. Week 1 — Prove the Mechanism + Core Domain

2.1 Objective

By the end of this week, you should have (a) a confirmed working WebSocket/STOMP round-trip, and (b) the basic authenticated domain (users, roles, orders as data) — but these two things do not need to be connected to each other yet.

2.2 Daily Breakdown

Day	Focus
Day 1	Environment setup checklist complete. Spring Boot project initialized with WebSocket starter. Read/skim Spring's WebSocket + STOMP documentation.
Day 2	Minimal WebSocketConfig working: one endpoint, one broker prefix. Manually test sending/receiving a message with a hardcoded topic using Postman or a simple test client.
Day 3	If Day 2's mechanism works: begin User entity, registration/login, JWT — reused pattern from prior project. If it does NOT work: continue debugging WebSocket, this takes priority over everything else this week.
Day 4	Order entity + state machine (validated transitions map + StageHistory), reusing the pattern already learned in the previous project.

Day	Focus
Day 5	DriverProfile entity (status + last-known lat/lng, updated via plain REST for now). Role-based + ownership-based guards on all REST endpoints written so far.
Day 6	Buffer day: catch up on anything slipped. Write Week 1 entries in NOTES.md documenting any trade-offs made.
Day 7	Rest or light review. Re-read Week 2 plan before starting.

2.3 Week 1 Deliverables

- ☐ A confirmed, demonstrable WebSocket round-trip (message sent from one client, received by another) — even with hardcoded/test topics.
- ☐ User registration, login, and JWT issuance working for Customer, Driver, and Dispatcher roles.
- ☐ Order entity with a validated state machine (illegal transitions rejected, legal ones recorded in history).
- ☐ DriverProfile entity storing status and last-known location, updatable via REST.

2.4 Self-Check Gate Before Moving to Week 2

- ☐ Can I send a WebSocket message from a script/tool and see it arrive on a subscribed client, reliably, more than once?
- ☐ Can I log in as two different users and confirm their JWTs are distinct and correctly scoped?
- ☐ Does my Order state machine reject an illegal transition (e.g., CREATED straight to DELIVERED)?

If the WebSocket mechanism is still not working reliably by the end of Week 1, this is the moment to invoke the fallback discussed in the Development Guide's Risks section (simplify the transport) rather than carry an unresolved foundational risk into Week 2.

3. Week 2 — Merge Real-Time Into the Domain

3.1 Objective

By the end of this week, the WebSocket mechanism from Week 1 and the domain model from Week 1 become one system: authenticated drivers send real location updates tied to real orders, and only the correct, authorized customer receives them.

3.2 Daily Breakdown

Day	Focus
Day 8	Wire authentication into the WebSocket handshake (JWT passed and validated at STOMP CONNECT time).
Day 9	Implement per-order topics (/topic/order/{orderId}) and enforce authorization at STOMP SUBSCRIBE time — test explicitly with two customer accounts.
Day 10	Integrate Redis to store latest-known driver location per order, replacing any in-memory placeholder.
Day 11	Implement the Haversine-based nearest-driver matching logic and wire it into order creation.
Day 12	Implement ETA calculation and include it in each location broadcast. Implement the async location history writer (confirm it does not block the broadcast).
Day 13	Buffer day: integration testing of the full flow end-to-end (create order → assign → driver sends location → customer receives it). Update NOTES.md.
Day 14	Rest or light review. Re-read Week 3 plan before starting.

3.3 Week 2 Deliverables

- ☐ A customer can create an order and be automatically matched with the nearest available driver.
- ☐ A driver's real-time location updates are broadcast only to the customer who owns that specific order.
- ☐ A second customer account, when tested, cannot subscribe to or receive another customer's order updates.
- ☐ Latest driver location is served from Redis; full history is durably and asynchronously logged.
- ☐ ETA is calculated and delivered alongside each location broadcast.

3.4 Self-Check Gate Before Moving to Week 3

- ☐ Have I actually tested the cross-customer isolation with two real accounts, not just assumed it works?
- ☐ Does the location broadcast still arrive promptly (under ~1 second) with the full pipeline (Redis + async history) attached?
- ☐ Can I explain, out loud, why location history writing is asynchronous and why Redis holds the live value?

4. Week 3 — Operational Features & Hardening

4.1 Objective

By the end of this week, Waypoint behaves like an operated system, not just a happy-path demo: it handles failures gracefully, resists abuse, and gives a dispatcher operational visibility.

4.2 Daily Breakdown

Day	Focus
Day 15	Dispatcher overview endpoint (active orders by stage, driver counts), cached with a short TTL using the pattern learned previously.
Day 16	Rate limiting on order creation and login endpoints using Redis counters; proper 429 responses.
Day 17	Driver disconnect handling: detect a dropped WebSocket session mid-order and flag the order for dispatcher attention.
Day 18	Build the driver simulator script — smooth simulated movement between two real coordinates, not random jitter.
Day 19	Unit tests for state machine transitions and matching logic; integration tests for REST endpoints.
Day 20	Buffer day: fix issues surfaced by testing. Update NOTES.md with any new trade-offs (e.g., simplifications in disconnect detection).
Day 21	Rest or light review. Re-read Week 4 plan before starting.

4.3 Week 3 Deliverables

- ☐ Dispatcher can view a live, cached operational overview of orders and drivers.
- ☐ Order creation and login are protected against abusive request bursts.
- ☐ A dropped driver connection mid-order results in the order being flagged, not silently stuck forever.
- ☐ A working driver simulator produces a visually convincing moving trajectory.
- ☐ A meaningful test suite exists covering the state machine and the matching logic at minimum.

4.4 Self-Check Gate Before Moving to Week 4

- ☐ If I kill a simulated driver's connection mid-delivery, does the system notice within a reasonable time?
- ☐ Do my rate limiter tests show a real 429 response once the limit is exceeded?
- ☐ Do my tests actually run and pass locally right now, not just exist as files?

5. Week 4 — Testing, Deployment & Presentation

5.1 Objective

By the end of this week, Waypoint is deployed, documented, and ready to be demoed and discussed in an interview setting.

5.2 Daily Breakdown

Day	Focus
Day 22	Multi-stage Dockerfile for the Spring Boot app; docker-compose wiring app + PostgreSQL + Redis.
Day 23	GitHub Actions pipeline: lint → test → build → deploy, failing the build on test failure.
Day 24	Deploy to Railway or Render; configure environment variables in the cloud dashboard (never committed to source control).
Day 25	Swagger/OpenAPI documentation for REST endpoints; final security pass (auth, input validation, data scoping) across the whole system.
Day 26	Write the README (architecture overview, local run instructions, trade-offs distilled from NOTES.md). Record a short demo video/screen capture of the simulator + live tracking.
Day 27	Buffer day: fix any deployment issues, polish documentation, rehearse explaining the architecture out loud.
Day 28	Final review against the Definition of Done in the Development Guide. Rest.

5.3 Week 4 Deliverables

- ☐ Fully Dockerized application with a working docker-compose stack.
- ☐ Green GitHub Actions pipeline that runs on every push and blocks on test failure.
- ☐ Live, publicly reachable deployment.
- ☐ Swagger/OpenAPI documentation for all public REST endpoints.
- ☐ A README that explains the architecture and includes the trade-offs distilled from NOTES.md.
- ☐ A short recorded demo showing live tracking in action.

5.4 Final Self-Check Gate — Project Complete

- ☐ Can two different logged-in customers each track their own driver simultaneously, with zero data leakage between them?
- ☐ Does the full order lifecycle work end-to-end through the deployed, public URL — not just locally?
- ☐ Can I whiteboard the architecture diagram from memory and explain at least three deliberate trade-offs I made?

- ☐ Is there a single command or short instruction set that lets someone else run this project locally from a fresh clone?

6. Overall Progress Snapshot

Fill this in at the end of each week as a running high-level summary — useful both for your own motivation tracking and as raw material for a portfolio write-up.

Week	Planned Theme	Actual Status	Notes / Deviations
1	Prove Mechanism + Core Domain		
2	Merge Real-Time Into Domain		
3	Operational Features & Hardening		
4	Testing, Deployment & Presentation		